

C++ 函数高级特性自学手册

模块一：内联函数 —— 提升执行效率的编译优化

1. 显式知识：定义与使用

- 语法规则：在函数声明或定义前加 `inline` 关键字，通常将函数定义直接放在原型位置（省略单独原型）。

```
inline double square(double x) { return x * x; } // 内联函数定义
```

- 核心特性：编译器用函数代码替换调用语句，避免函数调用的栈操作开销，但会增加代码体积。
- 使用场景：函数体简短（如几行代码）、被频繁调用（如循环内），不适合递归或长函数。

2. 隐式知识：设计本质与陷阱

- 与宏的本质区别：
 - 宏 (`#define SQUARE(x) x*x`) 是文本替换，可能导致逻辑错误（如 `SQUARE(3+1)` 展开为 `3+1*3+1`）。
 - 内联函数通过编译器处理，遵循语法规则，支持类型检查和参数计算（如 `square(3+1)` 正确计算为 16）。
- 编译器的“选择权”：

即使加 `inline`，编译器可能因函数过大、包含循环 / 递归等拒绝内联（视为“建议”而非强制）。

3. 实践建议

- 适用函数：简单运算（如求平方、判断奇偶）、访问器（getter）等。
- 避免场景：复杂逻辑（如多分支、循环）、递归函数（内联不支持递归）。

模块二：引用变量 —— 别名背后的内存机制

1. 显式知识：定义与基本操作

- 语法规则：用 `&` 声明，必须初始化（绑定到某变量），一旦绑定不可更改。

```
int a = 10;
int &b = a; // b是a的别名，与a共享内存
b++; // a和b的值均变为11
```

- 作为函数参数：实现“按引用传递”，函数可直接修改实参（替代指针的简洁写法）。

```
void swap(int &x, int &y) { int temp = x; x = y; y = temp; }
int main() {
    int m=3, n=5;
    swap(m, n); // m=5, n=3（直接修改实参）
}
```

2. 隐式知识：与指针的深层差异

特性	引用 (<code>int &</code>)	指针 (<code>int *</code>)
初始化	必须在声明时绑定，不可为 null	可先声明后赋值，可指向 null
内存地址	与绑定变量相同（无独立地址）	有独立地址，存储目标变量地址
操作符	直接使用 (<code>b = 5</code>)	需解引用 (<code>*p = 5</code>)
用途	函数参数、返回值（简化语法）	动态内存管理、数组遍历

3. 关键实践：const 引用的必要性

- 防止意外修改：用 `const` 修饰引用参数，确保函数不修改实参。

```
void print(const int &x) { x++; } // 编译错误：const引用不可修改
```

- **扩展参数兼容性**：`const` 引用可接收临时值（如表达式结果、不同类型转换后的值），非 `const` 引用不行。

```
double sum(const double &a, const double &b) { return a + b; }  
sum(3, 4.5); // 合法：临时转换为double
```

模块三：默认参数 —— 灵活调用的参数预设

1. 显式知识：语法与规则

- **设置方式**：在函数原型中为参数指定默认值，必须从右向左设置（右侧参数必须有默认值）。

```
// 原型：右侧参数有默认值，左侧可无  
void func(int a, int b = 2, int c = 3);
```

- **调用规则**：省略参数时使用默认值，不可跳过中间参数。

```
func(1); // 等价于 func(1, 2, 3)  
func(1, 5); // 等价于 func(1, 5, 3)  
func(1, , 4); // 错误：不能跳过中间参数
```

2. 隐式知识：设计逻辑与限制

- **为什么从右向左**：编译器按位置匹配参数，右侧默认值确保省略参数时无歧义。
- **声明与定义的一致性**：默认值只能在原型中设置（或定义中，但原型与定义不可重复设置）。
- **与函数重载的配合**：避免默认参数与重载函数特征标冲突（如 `func(int)` 与 `func(int, int=0)` 会导致二义性）。

模块四：函数重载 —— 同名函数的多态特性

1. 显式知识：特征标与匹配规则

- **核心条件**：函数名相同，参数列表（个数、类型、顺序）不同（特征标不同），返回类型无关。

```
void print(int x) { ... }  
void print(double x) { ... } // 重载（类型不同）  
void print(int x, double y) { ... } // 重载（个数不同）
```

- **调用匹配**：编译器根据实参类型 / 个数选择最匹配的版本，无法匹配时尝试隐式类型转换。

2. 隐式知识：易混淆场景解析

- **const 与非 const 参数**：视为不同特征标（适用于指针 / 引用）。

```
void func(int *x) { ... }  
void func(const int *x) { ... } // 合法重载
```

- **引用与非引用**：不视为不同特征标（`func(int)` 与 `func(int &)` 冲突）。
- **二义性处理**：若多个重载版本均匹配且无优先级，编译报错（需显式类型转换解决）。

模块五：函数模板 —— 通用代码的批量生成

1. 显式知识：定义与实例化

- **语法规则**：用 `template <typename T>` 声明，`T` 为类型参数，编译器根据实参类型生成具体函数（隐式实例化）。

```
template <typename T>  
T max(T a, T b) { return a > b ? a : b; }  
int main() {  
    max(3, 5); // 生成 int 版本  
    max(2.5, 1.8); // 生成 double 版本  
}
```

- **显式实例化**：手动指定类型，强制编译器生成特定版本。

```
template int max<int>(int, int); // 显式生成 int 版本
```

2. 隐式知识：具体化与重载解析

- **显式具体化**：为特定类型定制模板实现，优先级高于通用模板。

```
// 通用模板
template <typename T> void swap(T &a, T &b) { ... }
// 为结构体 Job 具体化
template <> void swap<Job>(Job &a, Job &b) {
// 仅交换 salary 和 floor 成员
}
```

- **编译器选择逻辑**：非模板函数 > 显式具体化 > 隐式实例化，多个模板版本时选择“最具体”的（如指针版本优先于通用版本）。

3. 实践技巧

- **避免过度通用**：模板需确保所有类型参数支持函数内操作（如 `+`、`>`），否则需限制类型或提供具体化。
- **与重载结合**：为模板添加非模板重载，处理特殊类型（如 `max(const char* a, const char* b)` 比较字符串）。

总结：特性选择决策树

1. **追求效率**：简短高频函数用**内联函数**，避免宏的安全问题。
2. **参数传递**：需修改实参用**引用**（简洁）或**指针**（灵活），只读参数加 `const`。
3. **多版本函数**：同功能不同参数用**重载**，同算法不同类型用**模板**。
4. **默认行为**：稳定的参数预设用**默认参数**，减少重载版本。

通过显式语法与隐式原理的结合，可灵活运用这些特性编写高效、易维护的 C++ 代码。

（注：文档部分内容可能由 AI 生成）